

Problem A. Waterloo Tag

Roger starts at building 1 and Troy releases students from a building x . For each x , we must maximize the time until the expanding wave of students reaches Roger. The graph is weighted, and both Roger and the students move continuously along edges.

Marks	Additional constraints
3	$N = 3, M = 2$
3	$N = 3, M = 3$
7	$v_1 = v_2 = 1$ and all edge lengths are 2
7	$N \leq 100, M \leq 200$
5	No additional constraints

Core observation

For a fixed source x , let $D_x[p]$ be the shortest distance from x to a point p of the graph. The students reach p at time $D_x[p]/v_2$. If Roger can reach p earlier, he can wait there and survive until that time. Thus every candidate answer is the student arrival time at some reachable vertex or interior point of an edge.

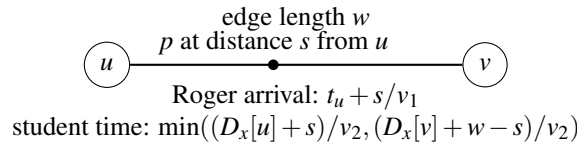


Figure 1. Meeting analysis on a single edge.

Subtask 1 – $N = 3$ and $M = 2$

The graph is a path with three vertices. For each possible x , there are only two edges and one middle vertex. Compute the student arrival time at every vertex and solve the one-dimensional meeting equation on each edge. Taking the maximum of these candidates gives the required fraction directly.

Subtask 2 – $N = 3$ and $M = 3$

The graph is a triangle. Use Floyd–Warshall or three runs of Dijkstra to get all vertex distances. The best final point is either a vertex or a point on one of the three edges. For an edge (u, v) of length w and a point at distance s from u , the student time is $\min((D_x[u] + s)/v_2, (D_x[v] + w - s)/v_2)$. This is piecewise linear, so the best feasible point is found by checking endpoints and the intersection of the two student waves.

Subtask 3 – equal speeds and all edges have length 2

Distances are twice unweighted graph distances. A vertex u is safe to stand on if $\text{dist}(1, u) < \text{dist}(x, u)$. BFS from 1 through safe vertices identifies where Roger can move without being caught. Each outgoing edge from a safe vertex gives either a safe endpoint or a final meeting point in its interior.

Subtask 4 – $N \leq 100$ and $M \leq 200$

For every x , run Dijkstra from x and keep the Dijkstra distances from 1. Then perform a Dijkstra-like reachability pass for Roger: a transition to a vertex is allowed only if Roger reaches that vertex before the students do. During the same pass, scan each edge from a reachable endpoint and solve the linear meeting equation to update the best answer.

Full solution

Precompute D_1 , the shortest distances from building 1. For each $x = 2, 3, \dots, N$, compute D_x by Dijkstra. Maintain the earliest safe arrival time of Roger at vertices. If Roger has safely reached u at time t_u , then along an edge (u, v)

of length w he is at distance s from u at time $t_u + s/v_1$, while the students arrive at $\min((D_x[u] + s)/v_2, (D_x[v] + w - s)/v_2)$. The feasible part of the edge is an interval starting at u ; its endpoint and the peak of the student-time function give all candidates. The total complexity is $O(N(M \log N + M))$.

Problem B. Melborp

We are given the array B and must reconstruct any array A with values in $[1, N]$ such that $B[i]$ counts the subarrays containing i in which $A[i]$ is a minimum.

Marks	Bounds / extra condition
2	$N \leq 8$
3	$N \leq 5000$ and original A is a permutation
5	$N \leq 3 \cdot 10^5$ and original A is a permutation
5	$N \leq 3 \cdot 10^5$
5	$N \leq 5 \cdot 10^6$ and original A is a permutation
5	$N \leq 5 \cdot 10^6$

Core observation

For a fixed valid A , let L_i be the closest index to the left of i with $A[L_i] < A[i]$, or 0 if none exists. Let R_i be the closest index to the right with $A[R_i] < A[i]$, or $N + 1$ if none exists. A subarray containing i has $A[i]$ as a minimum exactly when it stays inside (L_i, R_i) . Therefore $B[i] = (i - L_i)(R_i - i)$.

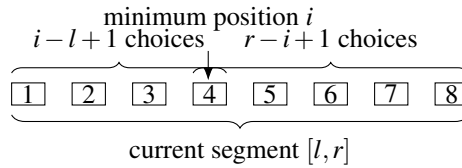


Figure 2. $B[i] = (i - l + 1)(r - i + 1)$; removing minima splits the gaps recursively.

Subtask 1 – $N \leq 8$

Brute force all candidate arrays A . For each candidate, compute its B array by enumerating all subarrays and all positions inside them. Since a solution exists, the first matching A can be printed.

Subtask 2 – $N \leq 5000$, original A is a permutation

In a permutation, every segment has a unique minimum. In a current segment $[l, r]$, the minimum position i is the unique index satisfying $B[i] = (i - l + 1)(r - i + 1)$. Assign the next smallest value to i , then recurse on $[l, i - 1]$ and $[i + 1, r]$. A linear scan of every segment is enough.

Subtasks 3 and 5 – permutation with large N

The same Cartesian-tree reconstruction is used, but the root of each segment must be found quickly. For each i , all possible segments $[l, r]$ for which i could be the minimum come from factor pairs $xy = B[i]$, with $l = i - x + 1$ and $r = i + y - 1$. The valid segments form a laminar family. Build this family with a stack or interval buckets, then assign increasing values by depth.

Subtask 4 – equal values allowed

When equal minima appear, several positions may have the same maximal segment. They can all receive the same value. After those positions are removed, the remaining gaps are solved independently with larger values. Thus the permutation reconstruction becomes a reconstruction of a laminar forest of intervals, where equal intervals correspond to equal values.

Full solution

Enumerate candidate intervals from divisor pairs of $B[i]$, keep only intervals contained in $[1, N]$, and organize the valid candidates into a laminar interval tree. Process the tree from the root downward. All positions whose chosen maximal interval is the current node receive the current value; child intervals are processed with larger values. Use 64-bit integers for $B[i]$. The implementation is near-linear apart from divisor enumeration and can be made suitable for N up to $5 \cdot 10^6$ with compact arrays and skip pointers.

Problem D. Asymmetry

The grid has even M . On one move a player changes one cell in an unmarked column and then marks the entire column. Alice minimizes the final horizontal asymmetry, and Bob maximizes it.

Marks	N	M	K
2	$N = 1$	$2 \leq M \leq 2000$	$1 \leq K \leq 10^9$
3	$N \leq 2000$	$M = 2$	$K = 1$
3	$N \leq 2000$	$M = 2$	$K \leq 10$
3	$N \leq 2000$	$M = 2$	$K \leq 10^9$
4	$N \leq 2000$	$M \leq 2000$	$K = 1$
4	$N \leq 2000$	$M \leq 2000$	$K \leq 10$
6	$N \leq 2000$	$M \leq 2000$	$K \leq 10^9$

Core observation

Consider a mirrored pair of columns $(j, M - j + 1)$. If Alice opens a pair, Bob never does worse by immediately taking its mirror column; if he instead opens another pair, Alice can answer there. Thus the game decomposes into independent mirrored pairs where Alice moves first in one column and Bob replies in the other.

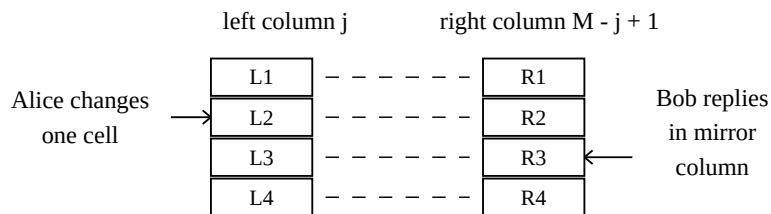


Figure 3. A mirrored column pair is solved as an independent two-move game.

For one pair, write the two column values in row i as L_i and R_i , and let $d_i = |L_i - R_i|$. If Bob changes the right cell in row i while the left cell is still L_i , he can make the row contribution $\max(L_i, K - L_i)$. The gain over the original row is $\max(L_i, K - L_i) - d_i$; the left-side gain is symmetric.

Subtask 1 – $N = 1$

Each mirrored pair is a single row. Alice changes one side to a value y , and Bob changes the other side to either 0 or K . The best Alice can do is choose y as close as possible to $K/2$, so each pair contributes $\lceil K/2 \rceil$. The answer is $(M/2)\lceil K/2 \rceil$.

Subtasks 2 and 3 – $M = 2$ and small K

There is only one mirrored pair. Enumerate Alice's first move: side, row, and value. For each choice, Bob's best response is either the mirrored cell in the same row or the best different row according to the precomputed gains. For $K = 1$ this is linear; for $K \leq 10$ it is $O(NK)$.

Subtask 4 – $M = 2$ and large K

For a fixed Alice side and row, only one integer y remains. If Alice changes the left cell of row a , Bob either replies in the same row, giving $\max(y, K - y)$, or in a different row, giving $|y - R_a| + C$, where C is the best different-row

gain. Alice minimizes $\max(\max(y, K - y), |y - R_a| + C)$; this is solved by interval intersection or a small binary search on the answer.

Subtasks 5 and 6 – many column pairs with small K

Apply the one-pair solution independently to each mirrored pair. For $K = 1$ the formula is simple; for $K \leq 10$ the value enumeration is small. Summing over all pairs gives $O(NM)$ or $O(NMK)$.

Full solution

For every mirrored pair, precompute the largest and second-largest left and right gains so that the best different-row response can be queried in $O(1)$. Scan every row twice, once for Alice changing the left cell and once for the right cell, using the interval formula from subtask 4. The minimum over these choices is the value of the pair. Add the values of all $M/2$ pairs. The full solution runs in $O(NM)$ time and uses $O(N)$ extra memory for one pair at a time.